Esercitazione 4: Breath First Search

Giacomo Paesani

April 8, 2026

Esercizio 1. Sia $G = (V, E)$ un grafo diretto, un vertice $u \in V$ è detto *principale* se per ogni vertice $v \in V$ esiste un cammino diretto da u a v. Fornire un algoritmo in pseudo-codice che, dato un grafo diretto G, determina tutti e soli i vertici principali di G in tempo $\mathcal{O}(|V| + |E|)$.

Soluzione 1. Per risolvere questo algoritmo, dobbiamo prima ricordare cosa è una componente fortemente connessa: sia $G = (V, E)$ un grafo diretto, allora una componente fortemente connessa di G è un sottoinsieme di vertici $U \subseteq V$ massimale tale che per ogni due vertici $u, v \in U$ allora esiste sempre un cammino da u a v e un cammino da v ad u.

Supponiamo inizialmente che G sia aciclico e che quindi ammette ordinamenti topologici per i suoi vertici. Sia $W \subseteq V$ l'insieme di tutti i vertici di G con grado entrante pari a 0: questi sono tutti i possibili candidati ad essere vertici principali. Ricordiamo che se G è aciclico, allora $|W| \geq 1$. Se l'insieme W contiene almeno due vertici w_1 e w_2 allora G non ammette vertici principali: infatti non esiste alcun cammino da w_1 a w_2 perché w_2 non ha alcun arco entrante e non esiste alcun cammino da w_2 a w_1 perché w_1 non ha alcun arco entrante.

Finalmente consideriamo il caso $W = \{w\}$. Dimostriamo che w è l'unico vertice principale di G. Sia v un qualsiasi vertice di G, e dimostriamo che esiste un cammino diretto da w a v. Supponiamo per assurdo che non esiste tale cammino. Consideriamo le due seguenti opzioni: se v non ha archi entranti, allora $v = w$ perché $W = \{w\}$; se invece v ha un arco entrante (u, v), allora non esiste neanche un cammino da w ad u. Iterando questo procedimento otteniamo due situazioni differenti: se tornando indietro con gli archi entranti incontriamo w, allora abbiamo ottenuto il cammino da w a v; altrimenti, cioè se non troviamo w, allora prima o poi troveremo la

ripetizione di un vertice e quindi la conferma dell'esistenza di un ciclo diretto. Si conclude che w è l'unico vertice principale di G.

Algorithm 1 Algoritmo per calcolare tutti i vertici principali di un grafo diretto.

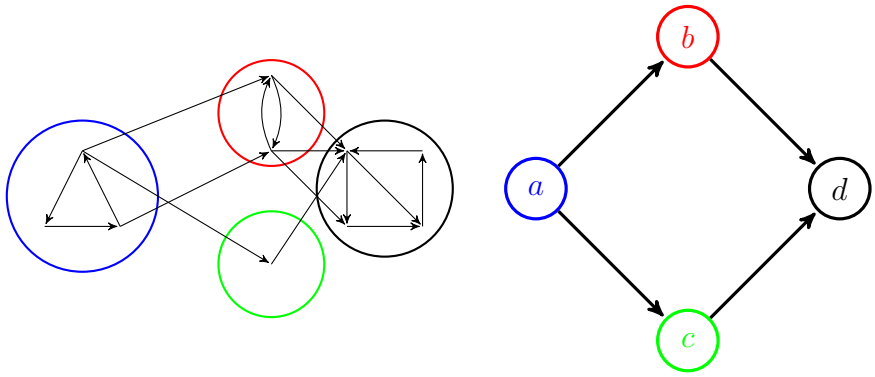
Input: grafo diretto $G = (V, E)$.
Output: insieme che contiene tutti i vertici principali di G.

```
1:  function PRINCIPAL(G)
2:      G^SCC = SCC(G)
3:      InDeg ↤ vettore dei gradi entranti di G^SCC, inizializzato a 0
4:      A ↤ insieme, inizialmente vuoto
5:      a = 0
6:      for u ∈ V(G^SCC) do
7:          for (v, u) ∈ E(G^SCC) do
8:              InDeg[u] = InDeg[u] + 1
9:      for u ∈ V(G^SCC) do
10:         if InDeg[u] = 0 ∧ a ≠ 0 then
11:             return ∅
12:         if InDeg[u] = 0 ∧ a = 0 then
13:             a = 1
14:             p = u
15:     for w ∈ p do
16:         A.add(w)
17:     return A
```

Consideriamo ora il caso più generale, cioè quello in cui G contiene cicli. Facciamo ora la seguente affermazione: siano v_1 e v_2 due vertici appartenenti alla stessa componente fortemente connessa di G, allora v_1 è un vertice principale di G se e solo se v_2 lo è. Infatti se da v_1 esiste cammino diretto per ogni altro vertice di G, allora tale cammino esiste anche da v_2 (basta usare il cammino da v_2 a v_1 come prefisso) e viceversa.

Grazie alla precedente affermazione possiamo adottare l'idea implementata nel Algoritmo 1: prima calcoliamo il grafo G^{SCC} delle componenti fortemente connesse di G con la chiamata alla funzione SSC(G) (Linea 2), esso è per sua natura aciclico, e su G^{SCC} possiamo adottare la strategia descritta per i grafi aciclici. Il ciclo **for** in Linea 6 serve a calcolare il grado entrante di ogni vertice di G^{SCC} e salvarlo nel vettore $InDeg$. La variabile a controlla il numero di vertici di G^{SCC} con grado entrante nullo, se ce n'è più di uno e

quindi a viene aggiornata più di una volta nel corso dell'algoritmo allora non vi sono vertici principali in G^{SCC} e quindi neanche in G (Linea 11). Supponiamo invece che a viene aggiornata una volta sola e quindi esiste un solo vertice u di G^{SCC} con grado entrante nullo. Allora u viene salvato nella variabile p (Linea 14) che rappresenta la componente fortemente connessa principale di G e quindi in A vengono aggiunti tutti i vertici di G in p (Linea 16), che quindi sono tutti e soli i vertici principali di G. $\square$



Esercizio 2. Sia $G = (V, E)$ un grafo non diretto, allora si definisce il *diametro* di G, $diam(G) = \max_{u,v \in V} dist(u, v)$, il massimo della distanza tra due qualsiasi vertici di G. Fornire un algoritmo in pseudo-codice che restituisca il diametro di un grafo G, nel caso in cui G sia un albero in tempo $\mathcal{O}(|V|)$.

Soluzione 2. Per dare una spiegazione convincente della soluzione proposta abbiamo bisogno della seguente affermazione.

Sia $G = (V, E)$ un albero e u un vertice di G. Se u è un vertice a distanza massima in una visita in profondità radicata in un nodo arbitrario r, allora esiste un altro vertice $v \in V$ tale che $dist(u, v) = diam(G)$. Supponiamo, per assurdo, che non esiste alcun vertice $v \in V$ tale che $dist(u, v) = diam(G)$. Allora esiste un cammino P di lunghezza massima tale che $\ell(P) > dist(u, v)$ per ogni $v \in V$. In particolare si può osservare che u non è un estremo e, visto che è una foglia, che non è un vertice interno di P; $u \notin P$.

Denotiamo con x e y i due estremi di P. Sia t il vertice di P più vicino a u e dividiamo il cammino P in due parti: P_x è il sotto-cammino di P tra x e t e P_y è il sotto-cammino di P tra y e t. Infine, denotiamo con P_u il cammino tra u e t. Consideriamo i due seguenti casi: se $\ell(P_u) = \ell(P_x) = \ell(P_y)$, allora

$dist(u, x) = dist(x, y) = diam(G)$, che è una contraddizione. Se altrimenti $\ell(P_u) > \min\{\ell(P_x), \ell(P_y)\} = \ell(P_x)$, allora $P_u P_y$ è un cammino più lungo di P, che che contraddice la massimalità della lunghezza di P.

L'Algoritmo 2 è una delle possibile soluzioni: il codice è inspirato alla BFS e l'idea è la seguente. Prima svolgiamo una BFS dell'albero scegliendo un qualsiasi vertice r come radice: otteniamo il vettore $Dist$ che registrata la distanza di ogni vertice di G da r. In seguito facciamo una seconda visita dell'albero radicata in uno dei vertici a distanza massima da r. Dopo questa ulteriore visita otteniamo il valore corretto del diametro dell'albero in input.

Durante la visita in profondità dell'albero G, nel vettore $Dist$, inizializzato a 0 (Linea 3), viene registrata la distanza da r. Il contatore c indica il numero di componente connesse di G: se ce n'è più di una allora il diametro di G è $+\infty$ e viene correttamente riportato in Linea 12.

Supponiamo ora che G ha esattamente una componente connessa, la chiamata DFS-Dist(G,r) (Linea 13) serve a trovare un vertice u (che in questo caso è una foglia) di G a distanza massima m dalla radice r. Allora eseguiamo una nuova ricerca in profondità di G con radice u (Linea 17). Al termine di della chiamata DFS-Dist(G,u), nella variabile m' viene registrato il diametro di G e riportato in Linea 18.

Concludiamo la soluzione a questo esercizio analizzando la complessità della soluzione proposta nel Algoritmo 2. La complessità è dominata dalle due ricerche in profondità consecutive e quindi $\mathcal{O}(|V| + |E|)$. Ricordando che se l'input è un albero, si ha che $|E| = |V| - 1$ e quindi la complessità de Algoritmo 2 è $\mathcal{O}(|V|)$. □

Esercizio 3. Fornire un algoritmo in pseudo-codice che, dato un grafo non diretto $G = (V, E)$ e due nodi u e v, restituisce tutti i nodi che hanno la stessa distanza da u e v in tempo $\mathcal{O}(|V| + |E|)$.

Soluzione 3. L'idea della soluzione è di implementare due BFS, una con radice u e l'altra con radice v, e per ogni vertice $w \in V$ confrontare la distanza da u a w con la distanza da v a w.

L'Algoritmo 3 è una delle possibili soluzioni: il codice è inspirato allBFS. Prima consideriamo il caso più semplice, cioè quello in cui i due vertici in input u e v sono uguali. Allora, grazie al ciclo **for** in Linea 8, si deduce banalmente che tutti i vertici hanno la medesima distanza da u e da v. L'effetto della chiamata alla funzione BFS in Linea 10 è quello che nel vettore d_1 vengono messe le distanze dei vertici di G da u. In maniera simile, l'effetto della chiamata alla funzione BFS in Linea 11 è quello che nel vettore d_2

Algorithm 2 Algoritmo per calcolare il diametro di un albero non diretto.

Input: grafo non diretto $G = (V, E)$.
Output: diametro di G.

```
global variables
    Visited ↤ vettore di |V| elementi, inizializzato a 0
    Dist ↤ vettore di |V| elementi, inizializzato a 0
    Q ↤ coda vuota
end global variables
function DIAMETER(G)
    c = 0
    for z ∈ V do
        if Visited[z]=0 then
            c = c + 1
            if c > 1 then
                return +∞
            (u, d)=BFS-DIST(G,z)
    for z ∈ V do
        Visited[z] = 0
        Dist[z] = 0
    (u′, d′)=BFS-DIST(G,u)
    return d′
function BFS-DIST(G,u)
    z = u
    max = 0
    Q.enqueue(u)
    while Q ≠ ∅ do
        v ↤ Q.dequeue()
        for w ∈ N(v) do
            if Visited[w] = 0 then
                Visited[w] = 1
                Dist[w] = Dist[v] + 1
                if Dist[w] > max then
                    max = Dist[w]
                    z = w
                Q.enqueue(w)
    return (z, max)
```

vengono messe le distanze dei vertici di G da v. Alla fine, è sufficiente confrontare le coordinate dei vettori d_1 e d_2 per determinare i vertici di G che sono equidistanti da u e da v. Questo completa la descrizione della funzione BFS-EQUIDISTANT.

Algorithm 3 BFS modificata per tutti i vertici equidistanti da due vertici.

Input: grafo $G = (V, E)$ e due vertici u e v.
Output: .

```
1: global variables
2:     Parent ↤ vettore di |V| elementi
3:     Color ↤ vettore di |V| elementi
4:     S ↤ insieme, inizialmente vuoto
5:     C coda, inizialmente vuota
6: end global variables
7: function BFS-EQUIDISTANT(G, u, v)
8:     if u = v then
9:         return V
10:    d1 =BFS(G,u)
11:    d2 =BFS(G,v)
12:    for w ∈ V do
13:        if d1[w] = d2[w] then
14:            S.add(w)
15:    return S
16: function BFS(G,z)
17:    dist ↤ vettore di |V| elementi, inizializzato a 0
18:    C.enqueue(z)
19:    while C ≠ ∅ do
20:        w = C.dequeue()
21:        for t ∈ Adj[w] do
22:            if dist[t] = 0 and t ≠ z then
23:                Parent[t] = w
24:                dist[t] = dist[w] + 1
25:                C.enqueue(t)
26:    return dist
```

La soluzione si conclude specificando che la funzione BFS in Linea 16 è inspirata dal codice della ricerca in ampiezza. □

Esercizio 4. Rimuovere un arco da un grafo (diretto o non diretto) modifica le distanze tra alcune coppie di vertici e ne lascia invariate alcune. Fornire un

algoritmo in pseudo-codice che, dato un grafo $G = (V, E)$ diretto, un vertice $s \in V$, un arco $(u, v) \in E$ e un vettore dei padri $Parent$, relativo ad una BFS effettuata su G con radice s, determina se la rimozione di (u, v) da G modifica le distanze da s in tempo $\mathcal{O}(|V|)$.

Soluzione 4. Facciamo prima delle considerazioni generali. Se l'arco (u, v) in input non è un arco dell'albero di visita della BFS con radice s allora la rimozione di tale arco non modifica le distanze. Consideriamo ora il caso in cui (u, v) è un arco dell'albero. Allora per risolvere l'esercizio è necessario capire se esiste un cammino in $G - (u, v)$ da s ad v della medesima lunghezza di quello da s a v in G passando per u.

Algorithm 4 Algoritmo che valuta se cambiano le distanze tra vertici a partire da un vertice rimuovendo un arco.

Input: grafo $G = (V, E)$, tre vertici s, u e v e un vettore $Parent$.
Output: `TRUE` se almeno una distanza da s a un vertice cambia e `FALSE` altrimenti.

```
global variables
    Dist ↤ vettore di |V| elementi , inizializzato a −1
end global variables
function Dist-Modif(G, s, u, v, Parent)
    if Parent[v] ≠ u then
        return FALSE
    for z ∈ V do
        CalcolaDistanza(G, z)
    for z ∈ V do
        if z ≠ u and Dist[z] = Dist[u] and (z, v) ∈ E then
            return FALSE
    return TRUE
function CalcolaDistanza(G,z)
    if Dist[z] = −1 then
        if Parent[z] = z then
            Dist[z] = 0
        else
            Dist[z] = 1+CalcolaDistanza(G, Parent[z])
    return
```

In particolare, si controlla se esiste un vertice z diverso da u tale che $Dist(s, u) = Dist(s, z)$ e $(z, v) \in E$: se esiste tale vertice allora esiste un cammino ulteriore rispetto a quello nell'albero di visita che preserva le dis-

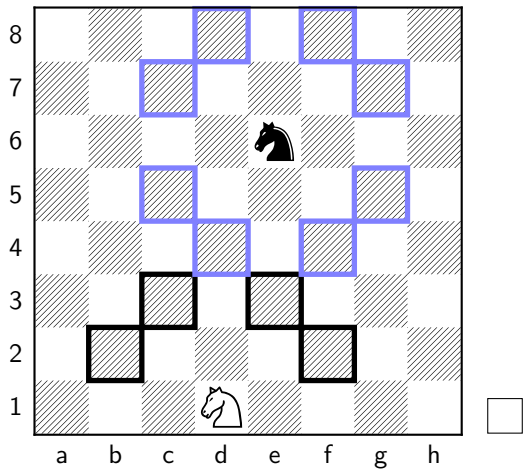
tanze, altrimenti le distanze vengono alterate. Una delle possibili soluzioni è presentata nel Algoritmo 4. La funzione CalcolaDistanza (Linee 13-19) sfrutta l'idea della programmazione dinamica e il vettore dei padri per calcolare le distanze di s dai vertici del grafo.

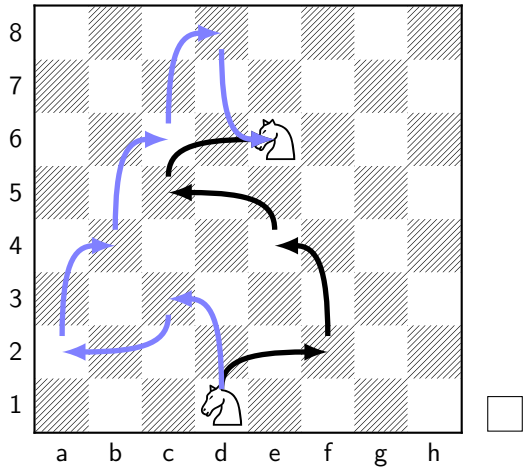
Se (u, v) non è un arco dell'albero di visita, allora in Linea 6 faccio ritornare correttamente `FALSE`. Supponiamo che (u, v) è un arco dell'albero di visita; allora con la funzione CalcolaDistanza di Linea 8 permette il calcolo delle distanze da s. Si scorre la lista dei vertici di G per controllare se esiste un vertice z adiacente a v diverso da u tale che $Dist(s, u) = Dist(s, z)$. Se tale vertice esiste allora viene riportato correttamente il valore `FALSE` in Linea 11. Altrimenti si deduce che ogni cammino di lunghezza minima in G da s a v usa l'arco (u, v) e allora l'algoritmo riporta `TRUE` in Linea 12.

Soffermiamoci, in fine, sul tempo di esecuzione dell'Algoritmo 4. Partendo da s e procedendo induttivamente sui suoi vertici a se adiacenti, le chiamate a CalcolaDistanza si risolvono in tempo $\mathcal{O}(|V|)$. Inoltre, per ogni vertice $z \in V$, verificare che z soddisfa le condizioni di Linea 10 richiede tempo costante $\mathcal{O}(1)$. Quindi la complessità della soluzione è $\mathcal{O}(|V|)$. □

Esercizio 5. Nel gioco degli scacchi il *cavallo* si muove nella seguente maniera: due caselle in orizzontale e una in verticale, o viceversa. Fornire un algoritmo in pseudo-codice che, data una scacchiera con N righe e M colonne, la posizione di partenza e quella di arrivo, calcola il numero minimo di turni necessari ad un cavallo di passare da una posizione all'altra in maniera che il tempo di esecuzione sia pari a $\mathcal{O}(M \cdot N)$.

Che modifiche sono necessarie se invece del cavallo si usa uno degli altri pezzi? Oppure se sono presenti degli altri pezzi statici ma non mangiabili?


8
7
6
5
4
3
2
1
a b c d e f g h


8
7
6
5
4
3
2
1
a b c d e f g h